

Making KrishiDukaan fast and affordable

What was slowing down the marketplace, how it affected farmers, sellers and our Firebase bill, what we changed, and how we stay fast as we grow.

Status: built and tested · not live yet

13–33 s

to open **one** Market page on the website (measured 2 October)

99%

of all database work was **reading** the same data again and again

~3,000

background jobs a day that started for **no reason**

The change in one picture

~~480~~ → **20**

records read to open one Market page on the website

~~4,222+~~ → **0**

records downloaded each time a farmer pauses while typing a search in the app

~~40~~ → **0**

background jobs started for every 20 products a farmer scrolls past

- 1 The whole story on one page
- 2 Six things to know first
- 3 What we measured
- 4 Why it was slow
- 5 The 7 problems and how we fixed them
- 6 Before and after, in numbers
- 7 How it works now
- 8 What happens as we grow
- 9 Rules for every new feature
- 10 Monthly health check
- 11 Going live
- 12 Still open, and words used

The whole story on one page

Think of a village shop. A farmer asks the price of urea.

Shop A — how KrishiDukaan worked

- 1. The shop phones **all 30 suppliers** and works out the best price.
- 2. It answers the farmer.
- 3. For the **next** farmer, it does all of it **again**.

Slow for every farmer. The phone bill grows with every new supplier and every new customer.

Shop B — how it works now

- 1. The shop keeps a **printed price list**.
- 2. When a supplier changes a price, it updates **that one line**.
- 3. Every farmer gets an instant answer from the list.

Fast for every farmer. The cost stays small, however many farmers ask.

The work now happens **once, when data changes** — not every time someone opens a screen.

The biggest changes at a glance

What	Before	After
Website Market page	Up to 480 records, in up to 12 trips one after another. 13–33 seconds	About 20 ready-made product cards
Typing a search in the app	The whole catalogue and every review, downloaded on each pause	Nothing downloaded: search runs on the phone
Lists of shops (cart, map, app)	5 whole collections downloaded	1–2 records
Each product a farmer scrolls past	Saved on the product, waking 2 background jobs	Saved in a small separate record. No jobs
Website server	In the USA, far from our database in India	In Mumbai, next to the database

Farmers

Pages read about 20 products instead of hundreds or thousands, so they should open much faster. Typing a search no longer uses their mobile data.

Sellers

View, click and call statistics keep working, with all past numbers kept. Shop changes appear within about 5 minutes.

Company

Far fewer reads and ~3,000 fewer background jobs a day: the bill follows real use, not catalogue size. Farmers' personal details stay private.

"Before" numbers come from the live consoles and Arjun's tests. "After" numbers are what the new code reads. Real speeds will be measured after launch.

Six things to know first

Everything in this report comes back to these six ideas.



A read = one record sent to us

Firebase charges for every record it sends.
Showing 20 products costs 20 reads.
Downloading the whole catalogue costs 4,222 reads, every single time.



A write = one record saved

Some saves automatically wake up background jobs (Cloud Functions). We pay for each job, even when it has nothing to do.



Distance adds waiting

Every trip between our server and the database takes time. USA to India and back: about 0.2 seconds, for every single trip.



Trips in a row add up

If a page makes 40 trips one after another, the waits add up: $40 \times 0.2 \text{ s} = 8 \text{ seconds}$ of travel alone.



A sleeping server is slow to wake

After a quiet period, the website server must start up first. The first visitor waits (about 30 seconds measured).



Farmers use phones

Many farmers have slow mobile networks and pay for data. Big downloads cost them time, data and battery.

Where a page's waiting time comes from

The four things that decide how fast a screen opens:

Cause	Example from KrishiDukaan	Fixed by this work?
How much data travels	App search downloaded 4,222 products + every review	Yes — product cards
How many trips in a row	Market page: up to 12 trips; search: up to 40	Yes — 1–2 trips
How far the server is	Server in the USA, database in India	Yes — both in Mumbai
Sleeping server	~30 s for the first visitor after a quiet period	Not yet — needs an always-on server (costs money)

What we measured

From the Firebase and Google Cloud consoles (early October 2026) and Arjun's tests on 2 October.

We read far more than we saved

Firestore usage in the period checked.



For every record saved, about **76 were read**. Reading is where the cost and the slowness were.

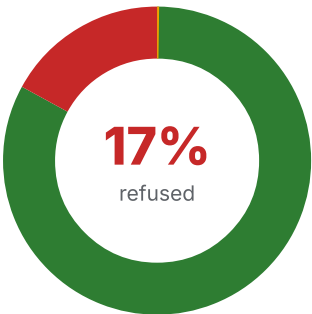
Background jobs ran 1,500 times for about 1 order

Cloud Function runs in 24 hours.



These two jobs listen for product changes. They were woken by **view counters** saved on products, not by real changes.

1 in 6 requests was refused

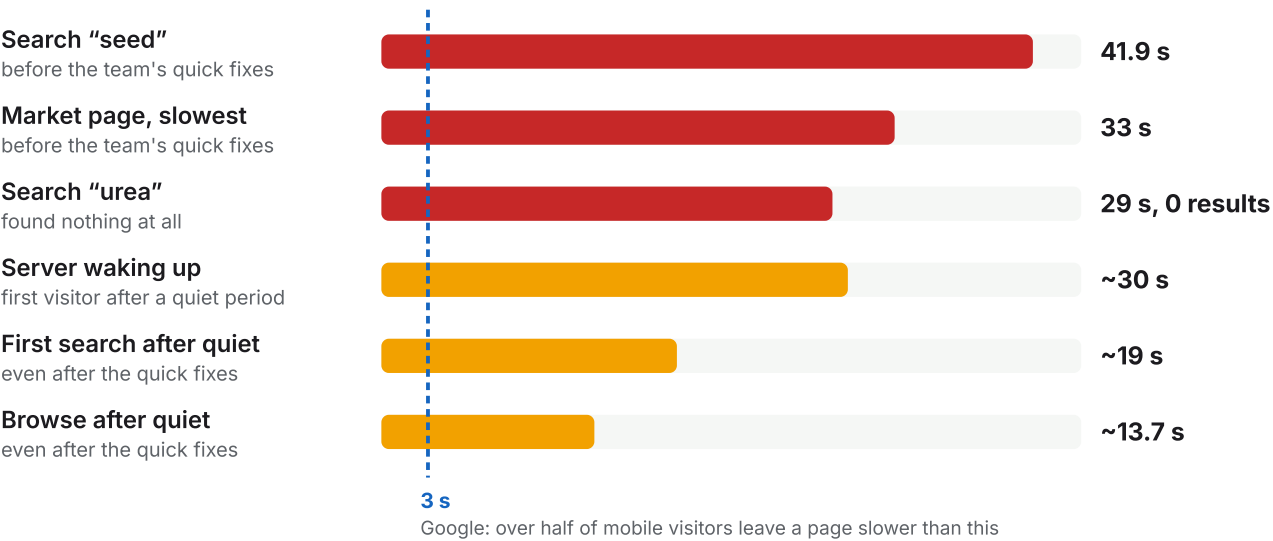


- ~1.7 million allowed
- 349,000 refused by the security rules
- 3,200 errors

A refused request is wasted work and usually a bug. Known causes: visitors who weren't signed in trying to save view counts, and farmers trying to tag someone.

How long things took (seconds)

Measured on the live site by Arjun on 2 October. Red: before the team's quick fixes. Amber: still there after the quick fixes, whenever the site had been quiet.



The quick fixes (a 60-second cache) helped busy times, but the **root cause** — combining thousands of records on every visit — was still there. This work removes it.

The single biggest query: the Google sitemap

Query insights, top query by load.

Query	Records per run	Runs in the window	Who runs it
products ... LIMIT 5000	4,222 (every product)	29	The sitemap we give Google

The query our teammate found

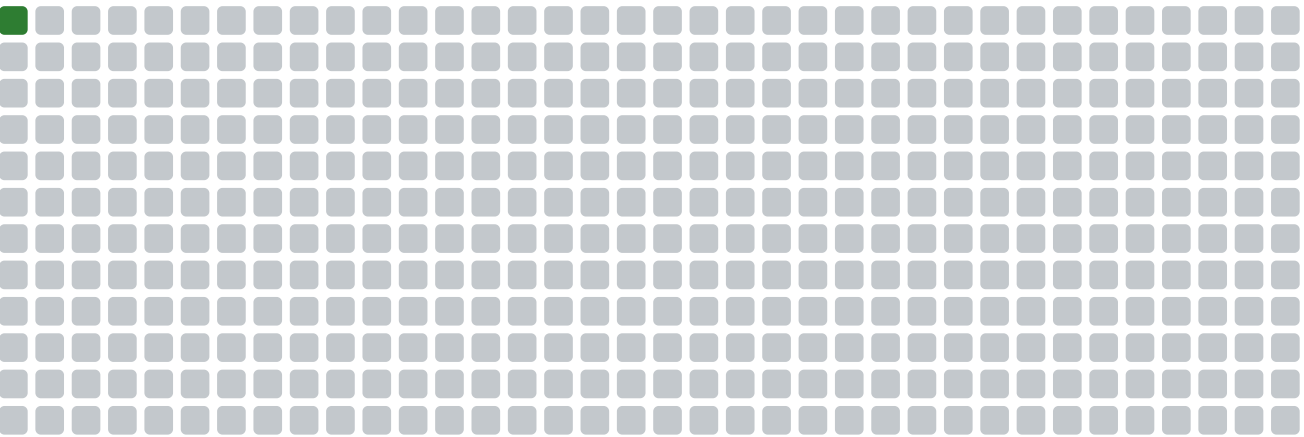
`COLLECTION /retailers ORDER_BY __name__ ASC` means: **download every retailer**, with no filter and no limit.

Six places in the code did this, usually together with four other whole collections. Before Sai's fix it also ran on every home page visit — the team's own estimate was about **20 million reads**.

Why it was slow

Reason 1 — downloading everything to show a little

To show **8 search suggestions**, the app downloaded all **4,222 product records**, plus every review. Each square below is about 10 records.



■ Downloaded ■ Actually shown to the farmer

And it did this on **every pause while typing**, on every category change, and when a product was opened from a shared link.

Reason 2 — many trips, one after another

BEFORE: website search for a rare word, up to 40 trips, one after another



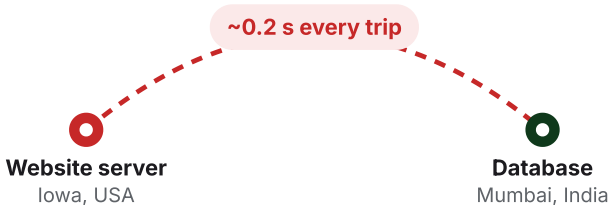
Each trip waits for the one before. With the server in the USA: $40 \times \sim 0.2 \text{ s} \approx 8 \text{ seconds}$ of travel alone.

AFTER: one search = 1-2 trips

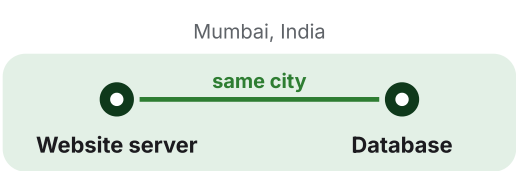


Reason 3 — the website server was on the other side of the world

BEFORE



AFTER



Moving the server to Mumbai was a one-line change. The ~ 30 -second wake-up after a quiet period is a separate problem (see page "Still open").

The 7 problems and how we fixed them

1

The same product was saved many times, and combined on every visit

Who felt it: **Farmers** **Sellers** **Firebase bill**

BEFORE

Every shop that sells a product gets its **own copy** of it — about 32 copies per product, 4,222 records in total. Each time anyone opened Market, searched, or opened a product, the system fetched all copies and every review, then worked out the price, sellers and rating.

AFTER

A background job builds **one ready-made product card** per product whenever a product or review changes. Screens just read the cards. Search uses a list of words for each card, including parts of words, Hindi names ("यूरिया") and common misspellings ("uria").

Result: Market page: about 20 cards instead of up to 480 records. App search while typing: 0 reads. Opening a product: 1 read. Cards show exactly what the old site showed (checked on 1,082 test cards).

For developers: `marketplaceCards`, `marketplaceSearch` · `functions/src/marketplace/`

2

Every product view was saved on the product, and woke up two jobs

Who felt it: **Sellers** **Firebase bill**

BEFORE

To count views for sellers' statistics, each product a farmer scrolled past was saved again. Every save woke two background jobs with nothing to do (~1,540 times a day each). Product records also grew bigger every day. Visitors who weren't signed in were refused.

AFTER

Counts are saved in a **separate small record** that wakes no jobs. The website sends them in groups. Visitors who aren't signed in aren't counted (their saves were refused anyway). Dashboards add old and new numbers, so no history is lost.

Result: About **3,000 fewer background jobs a day**. Product records stop growing. Fewer refused requests.

For developers: `productStats`, `storeStats`

3

Every list of shops downloaded five whole collections

Who felt it: **Farmers** **Firebase bill**

BEFORE

One shop's details are spread over 5 collections. To show any list of shops (cart, store map, shop pages, app store list, tagging), everything in all 5 was downloaded and matched up. This is the `/retailers` query our teammate found.

AFTER

A background job packs every shop's needed details into **1-2 records**, and refreshes them within about 5 minutes of any change. Screens keep their existing logic, so they show the same shops.

Result: 1-2 reads instead of five whole collections.

For developers: `storeDirectory` · `functions/src/stores/directory.ts`

4

Tagging someone downloaded every user

Who felt it: **Privacy** **Firebase bill**

BEFORE

Typing "@" to tag someone (website and app), the add-retailer form and the admin seller picker each downloaded **every user record** — including farmers' personal details. Most also downloaded every retailer.

AFTER

Shops come from the shop list above. People are found with a name search that returns **at most 10** records.

Result: Much cheaper, and farmers' personal details stay private. Farmers can now tag shops (before, their download was refused, so they could tag no one).

For developers: `app/reels/ReelComments.tsx`, `mobile user_tag_dialog.dart`

5

The website server was in the USA, the database in India

Who felt it: **Farmers**

BEFORE

Every request from the website server to the database travelled USA ↔ India: about 0.2 s per trip, and pages made many trips.

AFTER

Both website servers (shop and admin) now run in **Mumbai**, next to the database.

Result: Every trip gets much shorter. (The wake-up delay after quiet periods is separate.)

For developers: `firebase.json` → `region asia-south1`

6 The Google sitemap read every product

Who felt it: **Firestore bill**

BEFORE

The list of pages we give Google read all **4,222 product records** each time it was rebuilt — the single biggest query.

AFTER

It reads the product cards instead, and is rebuilt at most every 6 hours.

Result: Google gets the same list of pages, for far fewer reads.

For developers: `app/sitemap.ts`, `app/lib/seo/products-server.ts`

7 We had no real speed numbers

Who felt it: **Farmers**

BEFORE

We only knew speeds from our own computers, not from farmers' phones.

AFTER

Firebase Performance Monitoring records real load times on the website and in the app, by device and network.

Result: We can see the real effect of every change after launch.

For developers: App timers: `load_marketplace_cards`, `load_store_directory`

Already fixed by the team, before this work

- **Sai:** the home page loads about 10 top picks instead of the whole catalogue, and the shop list loads only when a screen needs it.
- **Arjun:** the Market feed is cached for 60 seconds, and search finds results anywhere instead of returning nothing.

Before and after, in numbers

Database work for common actions. Each row has its own scale: red is the old amount, green the new.



Every action, side by side

What the user does	Before	After
Opens the Market page (website)	Up to 480 records, up to 12 trips in a row	About 20 cards; repeats come from the cache
Searches (website)	The whole catalogue, held in server memory (~19 s when that memory was empty)	The search words for that word, plus one page of cards
Types a search (app)	Every product and every review, on each pause	0 — filters the cards already on the phone
Opens a shared product link (app)	Every product and every review	1
Opens the cart or store map (website)	5 whole shop collections + every product + every review	1–2 shop records + the product cards
Opens the store list (app)	5 whole shop collections	1–2 records
Scrolls past 20 products	20 saves on products + 40 background jobs (not signed in: 20 refused saves)	20 small saves sent together, 0 jobs (not signed in: nothing)
Types “@” to tag someone	Every user + every retailer	At most 10 user records + the shop list
Google sitemap is rebuilt	All 4,222 product records	One read per product card, at most every 6 hours

How it works now

Products

A product or review changes

a seller edits a price, a farmer posts a review



Original records

products, reviews — unchanged



Background job

runs within seconds



Product cards + search words

one ready-made card per product



Website and app

read only what they show

Shops

Shop details change

a shop, profile or shop review



Marked "needs update"

only if a field we show changed



Job every 5 minutes

rebuilds if needed



Shop list

all shops in 1–2 records



Cart, map, shop pages, app

1–2 reads

How quickly changes show up

Cards and the shop list are copies, so they update shortly after a change.

Price, stock, name or a new review

seconds

Shop details or a shop review

within ~5 min

A discount starting or ending on time

within 15 min

Website Market page (cache)

+1–2 min

Safety nets

Discount timer

Every 15 minutes, updates cards whose discount just started or ended.

Nightly repair

At 2:30 am, checks every card against the real data and fixes anything missed.

No overwriting

If two updates run at once, the older one is skipped.

Easy rollback

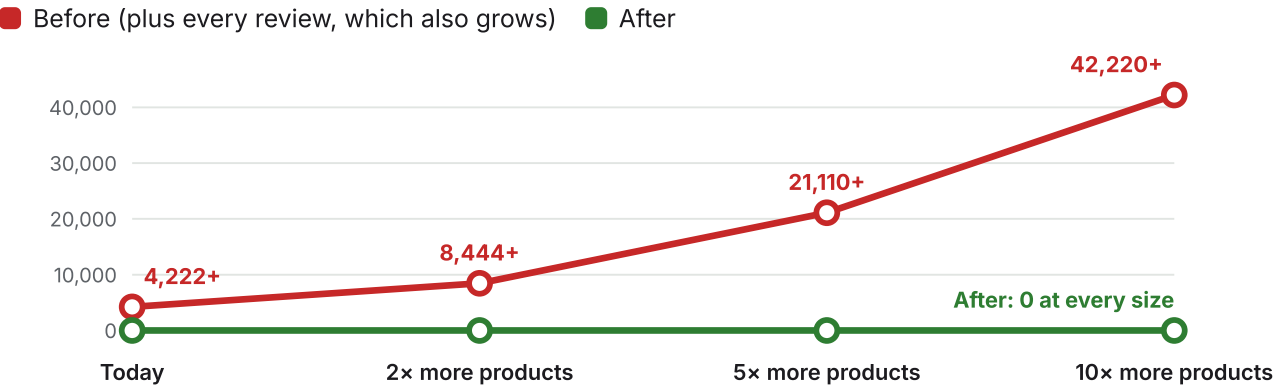
Original data is untouched, so old app versions keep working.

The trade-off: changes take a few seconds to a few minutes to appear, and building the cards costs some work when data changes. Data changes far less often than people look at it, so the total is much lower.

What happens as we grow

Records read for one pause while typing a search in the app

Before: every new product made every search more expensive. Now the cost of a search doesn't grow with the catalogue.



The app still loads all product cards once every 5 minutes. That is the first item in the table below.

The next limits, and what to do about them

What	Status	Watch for	Next step (Firebase / Google Cloud only)
The app (and some website pages) load all product cards	● Fine for now	The <code>load_marketplace_cards</code> timer gets slow, or there are a few thousand cards	Load 20 cards at a time, and only the cards a page needs
The shop list holds every shop in 1–2 records	● Fine for now	More than 2–3 records, or the <code>load_store_directory</code> timer gets slow	“Near me” search: load only shops close to the farmer (geohash)
Search matches words, parts of words and listed spellings	● Fine for now	Farmers type full sentences or spellings we didn't list	Google's AI search (Vertex AI) or Firestore vector search
One counter record per product	● Fine for now	“Contention” errors in the logs (about 1 save per second per record is the limit)	Split busy counters, or count views in Google Analytics
One shared daily visit counter for the whole site	● Plan it	Busy days	Remove it — Google Analytics already counts visits
Security rules look up extra records on each request	● Plan it	1–3 extra reads per request	Store phone and role in the login token (custom claims)
Sleeping website server (~30 s to wake)	● Decide soon	Slow first visits in Performance Monitoring	Keep 1 server always awake (costs money every hour)
Many copies of each product (the root cause)	● Plan it	The cards hide it for now	Long term: one record per product, plus one per seller offer

Rules for every new feature

Ask in every review: "How many records does this read if we have 10× more products, shops or users?"

If the answer grows with the size of a collection, stop and redesign it before it ships.

	Do	Don't
1	✓ Load about 20 items per page	✗ Download a whole collection for one screen
2	✓ Build combined answers once, in a background job	✗ Combine many records on every visit
3	✓ Keep counters in their own small records	✗ Save view counts on products, shops or users
4	✓ Run everything in Mumbai (asia-south1)	✗ Add servers in other regions without a reason
5	✓ Cache answers that are the same for everyone	✗ Mark public pages "never cache"
6	✓ Load details when the user opens them	✗ Preload data the screen doesn't show
7	✓ Search people with a small limit	✗ Send other users' personal details to a phone
8	✓ Add the index with a new query, and deploy it first	✗ Ship a query and wait for errors
9	✓ Treat every refused request as a bug	✗ Ignore the "Denies" number
10	✓ Measure before and after a change	✗ Guess

Worked example: "Show farmers the shops near them that sell urea"

THE SLOW WAY

Download every product, every shop and every review, then filter on the phone. Reads grow with every new product and shop, and the farmer waits for all of it.

THE FAST WAY

Open the urea **product card**: it already lists every seller with price and stock. The **shop list** (already loaded) gives each shop's location. 1 read, however big we get.

Monthly health check (15 minutes)

Where to look	What to check	Healthy	Warning sign
Firebase → Firestore → Usage	Reads, writes and refused requests per day	Reads rise with visitors, not with catalogue size. Refusals near zero	A jump after a release. Refusals rising
Google Cloud → Firestore → Query insights	Top queries by records scanned	Small numbers; queries return about what they scan	A query with no filter, or LIMIT 5000
Firebase → Functions → Usage	Runs per background job	Product jobs run about as often as real product edits	Thousands of runs with few real changes
Firebase → Functions → Logs	Errors in the card and shop-list jobs	No repeated errors	The same error every few minutes
Firebase → Performance	Website page load; the two app timers	Suggested target: under 2–3 s on 4G	Times rising month after month
Google Cloud → Billing → Budgets & alerts	A monthly budget with email alerts	Spending as expected	An alert fires

Monthly log

Print this page and fill it in. A trend over months tells you more than one day.

Month	Reads / day	Writes / day	Refused / day	Market load time	Notes

Going live

Do everything on **UAT** (the test project) first, then production. The order matters.

1

Security rules and indexes. Then wait until the 3 new indexes show **Enabled** in Firestore → Indexes (a few minutes).

2

Background jobs (Cloud Functions). Safe to deploy early: the live site doesn't use them yet.

3

Build the product cards and the shop list once, with the two scripts. Run each without `--write` first to preview.

4

Website. First check whether krishidukan.com is under **Hosting** or **App Hosting** in the Firebase console — it decides how the website deploys.

5

App. Test on a real phone, then publish. Older app versions keep working.

Website before step 3 = empty Market. The new website reads the cards and shop list, so they must exist first.

The deploy scripts stop early. `storage.rules` is missing from the repo, so `npm run deploy:prod` stops before deploying the background jobs. Add the live storage rules to the repo, or use the commands in the deploy guide that skip storage.

If something goes wrong: Website — Firebase console → Hosting → previous release → **Rollback**. Background jobs — delete them; they only write to the new collections, so nothing old breaks.

Exact commands: `docs/performance-rollout-2026-10.md`

After launch: watch for a week

- Firestore → Usage: daily reads should fall sharply.
- Functions: the two product jobs should drop from ~1,540 runs a day toward the number of real product edits.
- Performance: website page load and the two app timers.
- After a few quiet days, delete the old website servers in the USA.

Still open

- **Privacy:** any seller can still read any user's record. The sales dealers page and admin lists rely on it, so changing it needs its own careful test.
- **Security-rule lookups:** store phone and role in the login token (custom claims) so rules stop costing extra reads.
- **Sleeping website server:** decide whether to pay for one always-awake server.
- **Manufacturer's own discount** isn't shown in the lowest price when retailers also sell the product. The live site already behaves this way.
- **Daily visit counter:** remove it; Google Analytics already counts visits.
- **After most users update the app:** block view counters on products completely.
- **Repo:** add `storage.rules`, and move the test website server (UAT) to the test database's region.

Words used in this report

Word	Meaning
Record (document)	One item in the database, for example one product
Collection	A group of records, for example all products
Read / write	One record sent to us / one record saved. Both are billed
Query	A request for records that match a condition
Index	A lookup table that makes a query fast. Queries that filter and sort need one
Background job (Cloud Function)	Our code that runs on Google's servers, not on the user's device
Trigger	A background job that runs by itself when a record changes
Product card	A ready-made record with everything a screen needs for one product
Cache / CDN	A saved copy of an answer, reused for a short time / Google's servers that hand it out
Region	The city where a server or database runs. Ours: Mumbai (asia-south1)
Cold start	The extra time while a sleeping server wakes up
Security rules	The file that decides who may read or save which records
UAT	Our test project, where we try changes before real users see them